

Green Valley Estate - Database Design Phase Documentation

Group Members:

40584909 - NB Manganyi
40104648 - TD Maela
37853058 - TP Ndou
41719638 - JA Mokoena
38225964 - JP Poulou

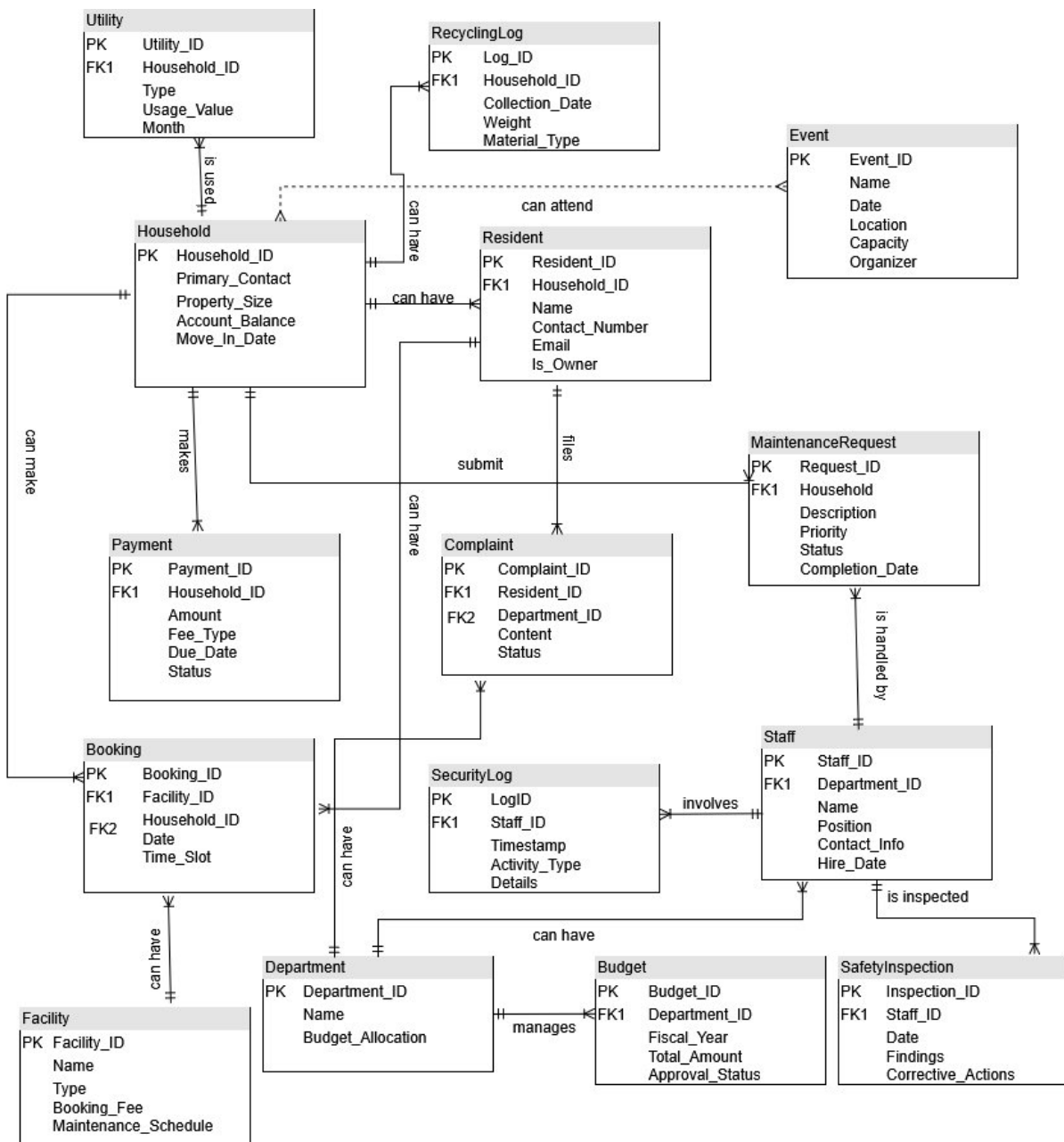
Table of Contents

Contents

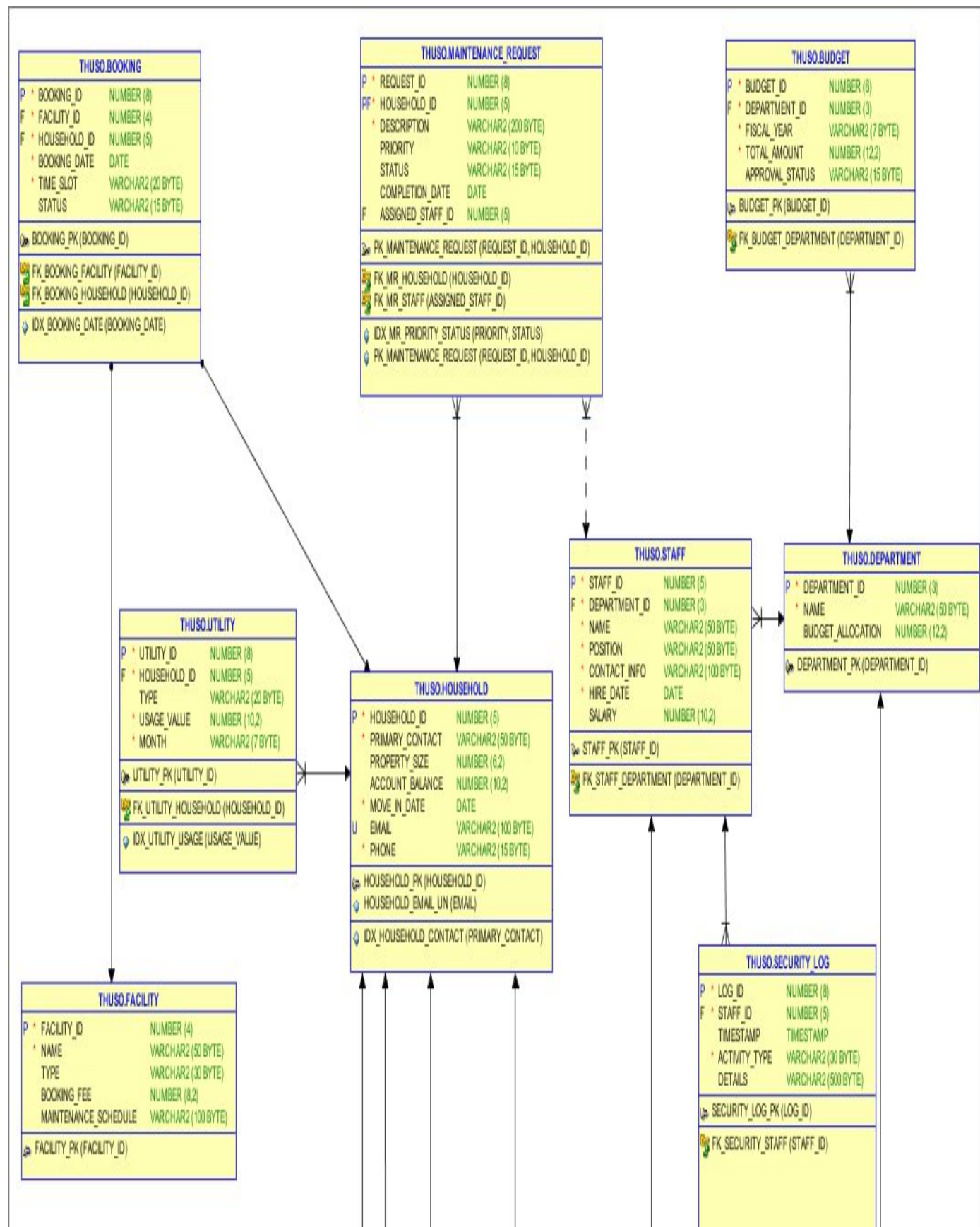
Green Valley Estate - Database Design Phase Documentation.....	1
Table of Contents.....	1
1. Entity Relationship Diagram (Crow's Foot Notation).....	2
2.Physical data model.....	3
3.Database objects.....	6
4.Queries.....	19

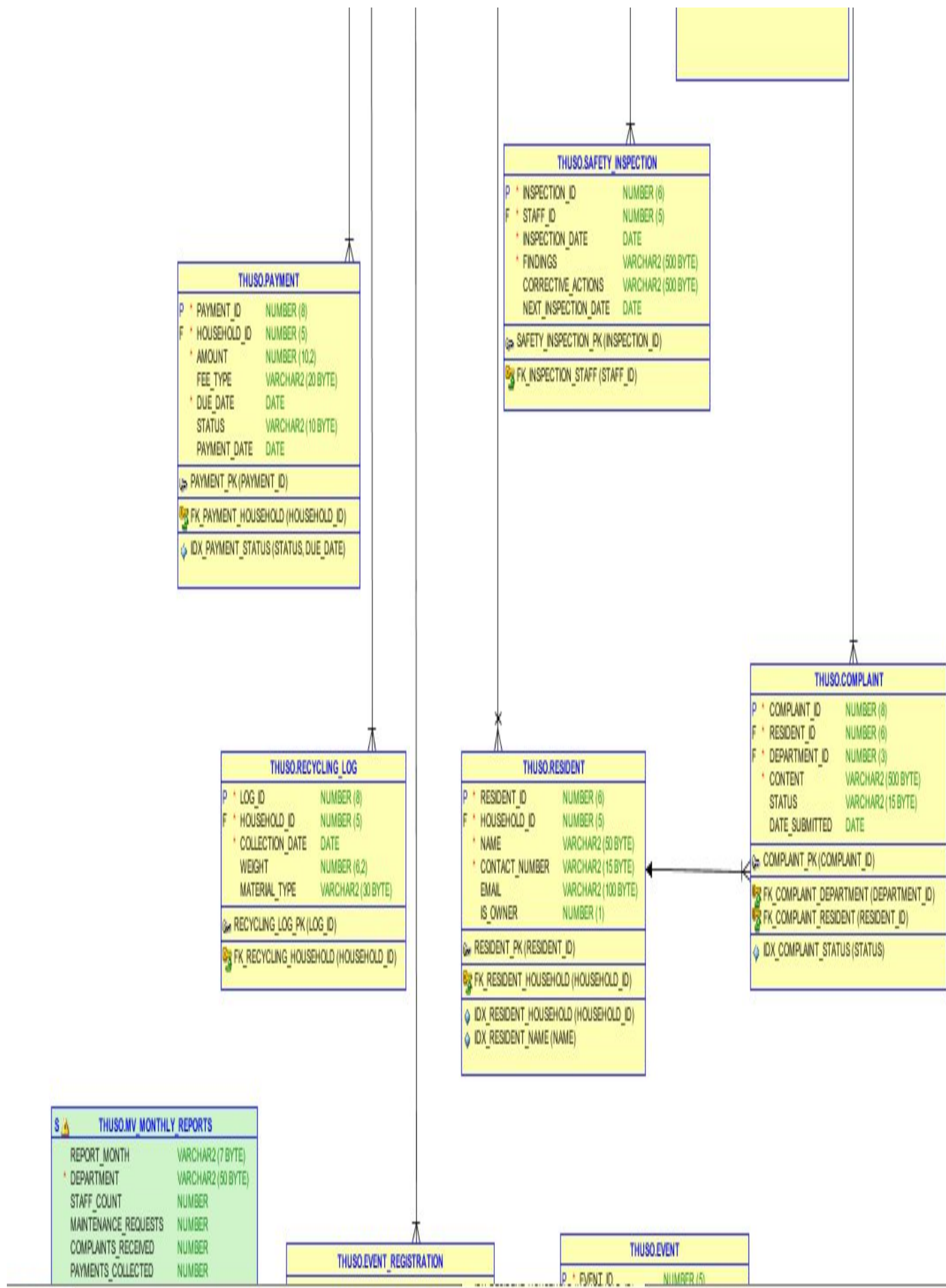
1. Entity Relationship Diagram (Crow's Foot Notation)

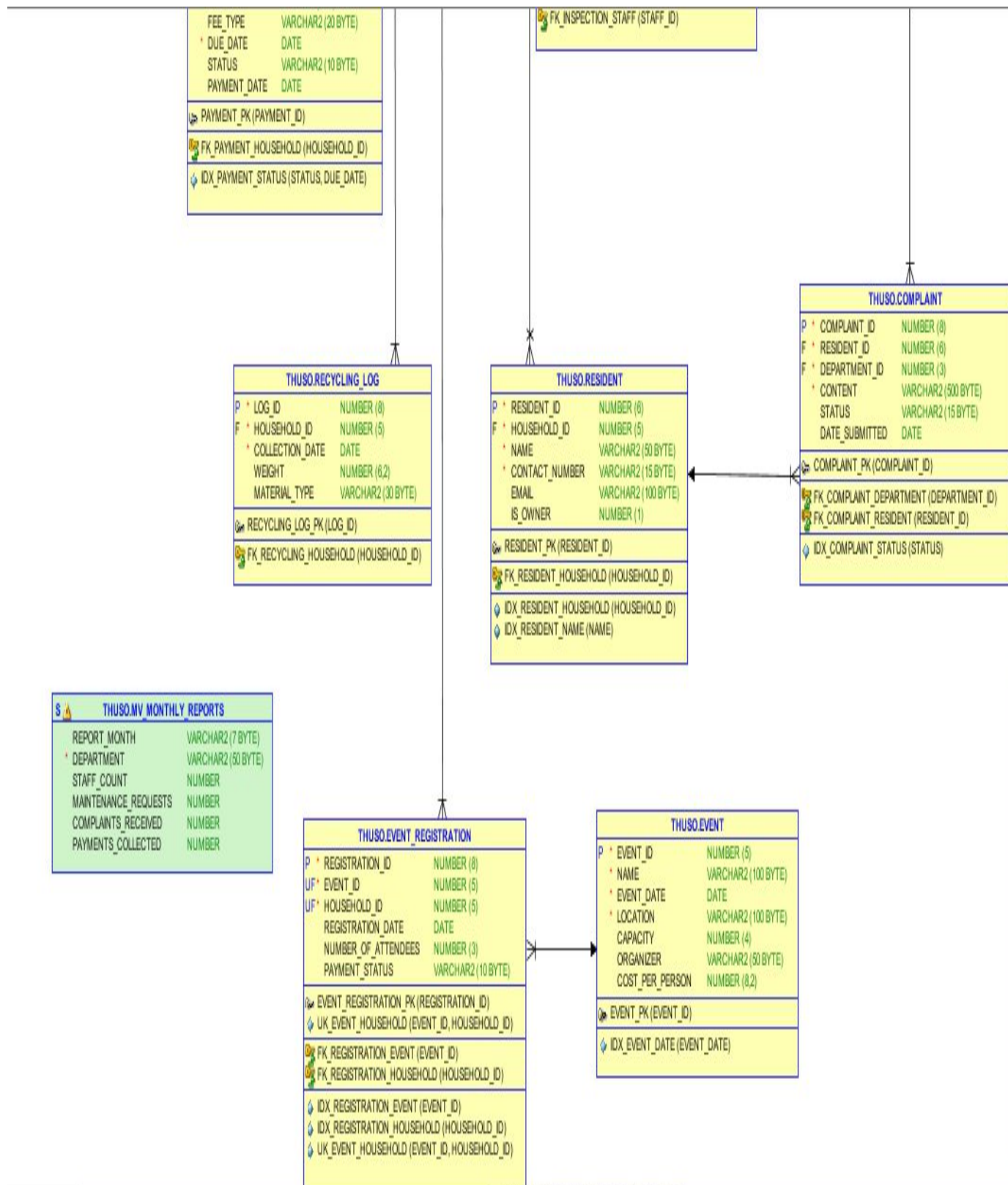
ERD Description: A conceptual diagram showing 15 entities and their relationships. Relationships include 1:M (Household → Payment) and M: N (Event → Registration).



2.Physical data model







3.Database objects

3.1 Tables

Department table

DEPARTMENT table with three columns: a numeric Department_ID as the primary key, a mandatory Name field limited to 50 characters, and a Budget_Allocation that must be zero or positive.

```
-- DEPARTMENT Table
CREATE TABLE DEPARTMENT (
    Department_ID NUMBER(3) PRIMARY KEY,
    Name VARCHAR2(50) NOT NULL,
    Budget_Allocation NUMBER(12,2) CHECK (Budget_Allocation >= 0)
);
```

Household table

HOUSEHOLD table with seven columns: a numeric Household_ID as the primary key, a mandatory Primary_Contact field, a positive Property_Size, an Account_Balance defaulting to zero, a required Move_In_Date, a unique Email, and a mandatory Phone number.

```
-- HOUSEHOLD Table
CREATE TABLE HOUSEHOLD (
    Household_ID NUMBER(5) PRIMARY KEY,
    Primary_Contact VARCHAR2(50) NOT NULL,
    Property_Size NUMBER(6,2) CHECK (Property_Size > 0),
    Account_Balance NUMBER(10,2) DEFAULT 0,
    Move_In_Date DATE NOT NULL,
    Email VARCHAR2(100) UNIQUE,
    Phone VARCHAR2(15) NOT NULL
);
```

Resident table

Records individual residents linked to households, with ownership flags and contact information.

```
-- RESIDENT Table
CREATE TABLE RESIDENT (
    Resident_ID NUMBER(6) PRIMARY KEY,
    Household_ID NUMBER(5) NOT NULL,
    Name VARCHAR2(50) NOT NULL,
    Contact_Number VARCHAR2(15) NOT NULL,
    Email VARCHAR2(100),
    Is_Owner NUMBER(1) DEFAULT 0 CHECK (Is_Owner IN (0,1)),
    CONSTRAINT fk_resident_household FOREIGN KEY (Household_ID)
        REFERENCES HOUSEHOLD(Household_ID) ON DELETE CASCADE
);
```

Staff table

Manages staff members with department assignments, positions, and salary information.

```
-- STAFF Table
CREATE TABLE STAFF (
    Staff_ID NUMBER(5) PRIMARY KEY,
    Department_ID NUMBER(3) NOT NULL,
    Name VARCHAR2(50) NOT NULL,
    Position VARCHAR2(50) NOT NULL,
    Contact_Info VARCHAR2(100) NOT NULL,
    Hire_Date DATE NOT NULL,
    Salary NUMBER(10,2) CHECK (Salary >= 0),
    CONSTRAINT fk_staff_department FOREIGN KEY (Department_ID)
        REFERENCES DEPARTMENT (Department_ID)
);
```

Payment table

Logs household payments with amount, type, due dates, and status tracking.

```
-- PAYMENT Table
CREATE TABLE PAYMENT (
    Payment_ID NUMBER(8) PRIMARY KEY,
    Household_ID NUMBER(5) NOT NULL,
    Amount NUMBER(10,2) NOT NULL CHECK (Amount > 0),
    Fee_Type VARCHAR2(20) CHECK (Fee_Type IN ('Security', 'Maintenance', 'Event', 'Facility', 'Event_Registration')),
    Due_Date DATE NOT NULL,
    Status VARCHAR2(10) DEFAULT 'Pending' CHECK (Status IN ('Pending', 'Paid', 'Late')),
    Payment_Date DATE,
    CONSTRAINT fk_payment_household FOREIGN KEY (Household_ID)
        REFERENCES HOUSEHOLD (Household_ID)
);
```

Security log table

Records security activities with timestamps and staff references.

```
-- SECURITY_LOG Table
CREATE TABLE SECURITY_LOG (
    Log_ID NUMBER(8) PRIMARY KEY,
    Staff_ID NUMBER(5) NOT NULL,
    Timestamp TIMESTAMP DEFAULT SYSTIMESTAMP,
    Activity_Type VARCHAR2(30) NOT NULL CHECK (Activity_Type IN ('Patrol', 'Incident', 'Access Control', 'CCTV Monitoring')),
    Details VARCHAR2(500),
    CONSTRAINT fk_security_staff FOREIGN KEY (Staff_ID)
        REFERENCES STAFF (Staff_ID)
);
```

Maintenance request table

Tracks repair requests with priorities, statuses, and staff assignments.

```
-- MAINTENANCE_REQUEST Table
CREATE TABLE MAINTENANCE_REQUEST (
    Request_ID NUMBER(8),
    Household_ID NUMBER(5) NOT NULL,
    Description VARCHAR2(200) NOT NULL,
    Priority VARCHAR2(10) CHECK (Priority IN ('Emergency', 'Urgent', 'Routine')),
    Status VARCHAR2(15) DEFAULT 'Submitted' CHECK (Status IN ('Submitted', 'Assigned', 'In Progress', 'Completed')),
    Completion_Date DATE,
    Assigned_Staff_ID NUMBER(5),
    CONSTRAINT pk_maintenance_request PRIMARY KEY (Request_ID, Household_ID),
    CONSTRAINT fk_mr_household FOREIGN KEY (Household_ID)
        REFERENCES HOUSEHOLD(Household_ID),
    CONSTRAINT fk_mr_staff FOREIGN KEY (Assigned_Staff_ID)
        REFERENCES STAFF(Staff_ID)
);
```

Event table

Stores event details including capacity, location, and costs.

```
-- EVENT Table
CREATE TABLE EVENT (
    Event_ID NUMBER(5) PRIMARY KEY,
    Name VARCHAR2(100) NOT NULL,
    Event_Date DATE NOT NULL,
    Location VARCHAR2(100) NOT NULL,
    Capacity NUMBER(4) CHECK (Capacity > 0),
    Organizer VARCHAR2(50),
    Cost_Per_Person NUMBER(8,2) DEFAULT 0 CHECK (Cost_Per_Person >= 0)
);
```

Event registration table

Manages event sign-ups with attendee counts and payment statuses.

```
-- EVENT_REGISTRATION Table (NEW)
CREATE TABLE EVENT_REGISTRATION (
    Registration_ID NUMBER(8) PRIMARY KEY,
    Event_ID NUMBER(5) NOT NULL,
    Household_ID NUMBER(5) NOT NULL,
    Registration_Date DATE DEFAULT SYSDATE,
    Number_Of_Attendees NUMBER(3) DEFAULT 1 CHECK (Number_Of_Attendees > 0),
    Payment_Status VARCHAR2(10) DEFAULT 'Pending' CHECK (Payment_Status IN ('Pending', 'Paid', 'Cancelled')),
    CONSTRAINT fk_registration_event FOREIGN KEY (Event_ID)
        REFERENCES EVENT(Event_ID),
    CONSTRAINT fk_registration_household FOREIGN KEY (Household_ID)
        REFERENCES HOUSEHOLD(Household_ID),
    CONSTRAINT uk_event_household UNIQUE (Event_ID, Household_ID)
);
```

Facility table

Lists estate facilities with booking fees and maintenance schedules.

```
-- FACILITY Table
CREATE TABLE FACILITY (
    Facility_ID NUMBER(4) PRIMARY KEY,
    Name VARCHAR2(50) NOT NULL,
    Type VARCHAR2(30) CHECK (Type IN ('Sports', 'Recreational', 'Community', 'Utility')),
    Booking_Fee NUMBER(8,2) DEFAULT 0 CHECK (Booking_Fee >= 0),
    Maintenance_Schedule VARCHAR2(100)
);
```

Booking table

Tracks facility reservations with time slots and status updates.

```
-- BOOKING Table
CREATE TABLE BOOKING (
    Booking_ID NUMBER(8) PRIMARY KEY,
    Facility_ID NUMBER(4) NOT NULL,
    Household_ID NUMBER(5) NOT NULL,
    Booking_Date DATE NOT NULL,
    Time_Slot VARCHAR2(20) NOT NULL,
    Status VARCHAR2(15) DEFAULT 'Confirmed' CHECK (Status IN ('Confirmed', 'Cancelled', 'Completed')),
    CONSTRAINT fk_booking_facility FOREIGN KEY (Facility_ID)
        REFERENCES FACILITY(Facility_ID),
    CONSTRAINT fk_booking_household FOREIGN KEY (Household_ID)
        REFERENCES HOUSEHOLD(Household_ID)
);
```

Complaint table

Logs resident complaints with department assignments and resolution statuses.

```
-- COMPLAINT Table
CREATE TABLE COMPLAINT (
    Complaint_ID NUMBER(8) PRIMARY KEY,
    Resident_ID NUMBER(6) NOT NULL,
    Department_ID NUMBER(3) NOT NULL,
    Content VARCHAR2(500) NOT NULL,
    Status VARCHAR2(15) DEFAULT 'Open' CHECK (Status IN ('Open', 'In Progress', 'Resolved', 'Closed')),
    Date_Submitted DATE DEFAULT SYSDATE,
    CONSTRAINT fk_complaint_resident FOREIGN KEY (Resident_ID)
        REFERENCES RESIDENT(Resident_ID),
    CONSTRAINT fk_complaint_department FOREIGN KEY (Department_ID)
        REFERENCES DEPARTMENT(Department_ID)
);
```

Budget table

Manages department budgets with approval status tracking

```
-- BUDGET Table
CREATE TABLE BUDGET (
    Budget_ID NUMBER(6) PRIMARY KEY,
    Department_ID NUMBER(3) NOT NULL,
    Fiscal_Year VARCHAR2(7) NOT NULL,
    Total_Amount NUMBER(12,2) NOT NULL CHECK (Total_Amount >= 0),
    Approval_Status VARCHAR2(15) DEFAULT 'Pending' CHECK (Approval_Status IN ('Pending', 'Approved', 'Rejected')),
    CONSTRAINT fk_budget_department FOREIGN KEY (Department_ID)
        REFERENCES DEPARTMENT(Department_ID)
);
```

Recycling log table

Records household recycling collections by material type.

```
-- RECYCLING_LOG Table
CREATE TABLE RECYCLING_LOG (
    Log_ID NUMBER(8) PRIMARY KEY,
    Household_ID NUMBER(5) NOT NULL,
    Collection_Date DATE NOT NULL,
    Weight NUMBER(6,2) CHECK (Weight > 0),
    Material_Type VARCHAR2(30) CHECK (Material_Type IN ('Paper', 'Plastic', 'Glass', 'Metal', 'Organic')),
    CONSTRAINT fk_recycling_household FOREIGN KEY (Household_ID)
        REFERENCES HOUSEHOLD(Household_ID)
);
```

Safety inspection table

Documents safety checks with findings and follow-up dates.

```
-- SAFETY_INSPECTION Table
CREATE TABLE SAFETY_INSPECTION (
    Inspection_ID NUMBER(6) PRIMARY KEY,
    Staff_ID NUMBER(5) NOT NULL,
    Inspection_Date DATE NOT NULL,
    Findings VARCHAR2(500) NOT NULL,
    Corrective_Actions VARCHAR2(500),
    Next_Inspection_Date DATE,
    CONSTRAINT fk_inspection_staff FOREIGN KEY (Staff_ID)
        REFERENCES STAFF(Staff_ID)
);
```

Utility table

Tracks household utility consumption by type and month.

```

-- UTILITY Table
CREATE TABLE UTILITY (
    Utility_ID NUMBER(8) PRIMARY KEY,
    Household_ID NUMBER(5) NOT NULL,
    Type VARCHAR2(20) CHECK (Type IN ('Water', 'Electricity', 'Gas')),
    Usage_Value NUMBER(10,2) NOT NULL CHECK (Usage_Value >= 0),
    Month VARCHAR2(7) NOT NULL,
    CONSTRAINT fk_utility_household FOREIGN KEY (Household_ID)
        REFERENCES HOUSEHOLD(Household_ID)
);

```

3.2 Indexes

1. **idx_household_contact**: Speeds up searches by household primary contact names.
2. **idx_resident_name**: Optimizes queries filtering by resident names.
3. **idx_resident_household**: Improves joins between residents and their households.
4. **idx_payment_status**: Enhances filtering by payment status and due dates.
5. **idx_mr_priority_status**: Accelerates maintenance request lookups by priority and status.
6. **idx_event_date**: Makes event date-based queries faster.
7. **idx_booking_date**: Improves performance for booking date searches.
8. **idx_complaint_status**: Speeds up filtering complaints by their status.
9. **idx_utility_usage**: Optimizes queries analysing utility consumption values.
10. **idx_registration_event**: Improves joins between event registrations and events.
11. **idx_registration_household**: Enhances joins between registrations and households.

```

-- =====
-- INDEXES FOR PERFORMANCE OPTIMIZATION
-- =====

CREATE INDEX idx_household_contact ON HOUSEHOLD(Primary_Contact);
CREATE INDEX idx_resident_name ON RESIDENT(Name);
CREATE INDEX idx_resident_household ON RESIDENT(Household_ID);
CREATE INDEX idx_payment_status ON PAYMENT(Status, Due_Date);
CREATE INDEX idx_mr_priority_status ON MAINTENANCE_REQUEST(Priority, Status);
CREATE INDEX idx_event_date ON EVENT(Event_Date);
CREATE INDEX idx_booking_date ON BOOKING(Booking_Date);
CREATE INDEX idx_complaint_status ON COMPLAINT(Status);
CREATE INDEX idx_utility_usage ON UTILITY(Usage_Value);
CREATE INDEX idx_registration_event ON EVENT_REGISTRATION(Event_ID);
CREATE INDEX idx_registration_household ON EVENT_REGISTRATION(Household_ID);

```


3.3 Views

vw_outstanding_payments: Shows unpaid overdue bills with calculated late fees (5% penalty) and total amounts due.

```
-- Current Outstanding Payments View
CREATE OR REPLACE VIEW vw_outstanding_payments AS
SELECT h.Household_ID, h.Primary_Contact, p.Payment_ID, p.Amount, p.Due_Date,
       ROUND(p.Amount * 0.05, 2) AS Late_Fee,
       p.Amount + ROUND(p.Amount * 0.05, 2) AS Total_Due
FROM HOUSEHOLD h
JOIN PAYMENT p ON h.Household_ID = p.Household_ID
WHERE p.Status = 'Pending' AND p.Due_Date < SYSDATE;
```

vw_active_maintenance: Displays all uncompleted repair requests with priority levels and days since submission.

```
-- 2. Active Maintenance Requests View
CREATE OR REPLACE VIEW vw_active_maintenance AS
SELECT mr.Request_ID, h.Household_ID, h.Primary_Contact, mr.Description,
       mr.Priority, mr.Status, s.Name AS Assigned_Staff,
       TRUNC(SYSDATE - mr.Completion_Date) AS Days_Open
FROM MAINTENANCE_REQUEST mr
JOIN HOUSEHOLD h ON mr.Household_ID = h.Household_ID
LEFT JOIN STAFF s ON mr.Assigned_Staff_ID = s.Staff_ID
WHERE mr.Status NOT IN ('Completed');
```

vw_event_participation: Provides event attendance statistics including registration counts and capacity percentages.

```
-- 3. Event Participation Summary View (UPDATED)
CREATE OR REPLACE VIEW vw_event_participation AS
SELECT
    e.Event_ID,
    e.Name,
    e.Event_Date,
    COUNT(er.Registration_ID) AS Registered_Households,
    SUM(er.Number_Of_Attendees) AS Total_Attendees,
    e.Capacity,
    ROUND(SUM(er.Number_Of_Attendees)/e.Capacity*100, 2) AS Participation_Percentage
FROM EVENT e
LEFT JOIN EVENT_REGISTRATION er ON e.Event_ID = er.Event_ID
GROUP BY e.Event_ID, e.Name, e.Event_Date, e.Capacity;
```


vw_utility_consumption: Summarizes household utility usage patterns with average, peak and minimum consumption values.

```
-- 4. Household Utility Consumption View
CREATE OR REPLACE VIEW vw_utility_consumption AS
SELECT h.Household_ID, h.Primary_Contact, u.Type,
       AVG(u.Usage_Value) AS Avg_Usage,
       MAX(u.Usage_Value) AS Peak_Usage,
       MIN(u.Usage_Value) AS Min_Usage
FROM HOUSEHOLD h
JOIN UTILITY u ON h.Household_ID = u.Household_ID
GROUP BY h.Household_ID, h.Primary_Contact, u.Type;
```

vw_household_events: Lists all event registrations per household with calculated total costs based on attendance.

```
-- 5. Household Event Registrations View (NEW)
CREATE OR REPLACE VIEW vw_household_events AS
SELECT
    h.Household_ID,
    h.Primary_Contact,
    e.Event_ID,
    e.Name AS Event_Name,
    e.Event_Date,
    er.Number_Of_Attendees,
    er.Payment_Status,
    (e.Cost_Per_Person * er.Number_Of_Attendees) AS Total_Cost
FROM HOUSEHOLD h
JOIN EVENT_REGISTRATION er ON h.Household_ID = er.Household_ID
JOIN EVENT e ON er.Event_ID = e.Event_ID;
```

3.4 Data on Tables

Department Data:

- Creates 6 core departments (Security, Maintenance, Finance, etc.) with budget allocations

```
-- Insert Departments
INSERT INTO DEPARTMENT VALUES (1, 'Security', 500000);
INSERT INTO DEPARTMENT VALUES (2, 'Maintenance', 450000);
INSERT INTO DEPARTMENT VALUES (3, 'Finance', 300000);
INSERT INTO DEPARTMENT VALUES (4, 'Community Services', 350000);
INSERT INTO DEPARTMENT VALUES (5, 'Health and Safety', 250000);
INSERT INTO DEPARTMENT VALUES (6, 'Environmental Management', 200000);
```

Household Registration: Five households are registered with their primary contacts (John Smith, Sarah Johnson, etc.), including property details (sizes from 180-320 sqm), account balances (ranging from -R750 to R1,250), and move-in dates spanning 2018-2021.

```
-- Insert Households
INSERT INTO HOUSEHOLD VALUES (seq_household_id.NEXTVAL, 'John Smith', 250.5, 0, TO_DATE('15-JAN-2020', 'DD-MON-YYYY'), 'john.smith@email.com', '0821234567');
INSERT INTO HOUSEHOLD VALUES (seq_household_id.NEXTVAL, 'Sarah Johnson', 180.0, 1250.50, TO_DATE('22-MAR-2019', 'DD-MON-YYYY'), 'sarah.j@email.com', '0839876543');
INSERT INTO HOUSEHOLD VALUES (seq_household_id.NEXTVAL, 'Michael Brown', 320.75, -750.25, TO_DATE('05-JUL-2021', 'DD-MON-YYYY'), 'michael.b@email.com', '0845678912');
INSERT INTO HOUSEHOLD VALUES (seq_household_id.NEXTVAL, 'Emily Davis', 195.25, 0, TO_DATE('30-NOV-2020', 'DD-MON-YYYY'), 'emily.d@email.com', '0723456789');
INSERT INTO HOUSEHOLD VALUES (seq_household_id.NEXTVAL, 'Robert Wilson', 275.0, 500.75, TO_DATE('14-FEB-2018', 'DD-MON-YYYY'), 'robert.w@email.com', '0812345678');
```

Resident Records: Seven residents are documented across these households, identifying primary owners (John Smith, Sarah Johnson, etc.) and secondary residents (Mary Smith, Lisa Brown), with complete contact details.

```
-- Insert Residents
INSERT INTO RESIDENT VALUES (seq_resident_id.NEXTVAL, 10001, 'John Smith', '0821234567', 'john.smith@email.com', 1);
INSERT INTO RESIDENT VALUES (seq_resident_id.NEXTVAL, 10001, 'Mary Smith', '0821234568', 'mary.smith@email.com', 0);
INSERT INTO RESIDENT VALUES (seq_resident_id.NEXTVAL, 10002, 'Sarah Johnson', '0839876543', 'sarah.j@email.com', 1);
INSERT INTO RESIDENT VALUES (seq_resident_id.NEXTVAL, 10003, 'Michael Brown', '0845678912', 'michael.b@email.com', 1);
INSERT INTO RESIDENT VALUES (seq_resident_id.NEXTVAL, 10003, 'Lisa Brown', '0845678913', 'lisa.b@email.com', 0);
INSERT INTO RESIDENT VALUES (seq_resident_id.NEXTVAL, 10004, 'Emily Davis', '0723456789', 'emily.d@email.com', 1);
INSERT INTO RESIDENT VALUES (seq_resident_id.NEXTVAL, 10005, 'Robert Wilson', '0812345678', 'robert.w@email.com', 1);
```

Staff : Six staff members are hired across departments - including security personnel (David Security, James Guard), maintenance workers (Paul Handyman, Mark Technician), a finance manager (Linda Accountant), and community manager (Susan Organizer), with salaries ranging from R25,000 to R40,000.

```
-- Insert Staff
INSERT INTO STAFF VALUES (seq_staff_id.NEXTVAL, 1, 'David Security', 'Security Manager', '0821112222', TO_DATE('10-JAN-2019', 'DD-MON-YYYY'), 35000);
INSERT INTO STAFF VALUES (seq_staff_id.NEXTVAL, 1, 'James Guard', 'Security Officer', '0834445555', TO_DATE('15-MAR-2020', 'DD-MON-YYYY'), 25000);
INSERT INTO STAFF VALUES (seq_staff_id.NEXTVAL, 2, 'Paul Handyman', 'Maintenance Supervisor', '0846667777', TO_DATE('05-JUL-2018', 'DD-MON-YYYY'), 32000);
INSERT INTO STAFF VALUES (seq_staff_id.NEXTVAL, 2, 'Mark Technician', 'Maintenance Technician', '0728889999', TO_DATE('20-SEP-2021', 'DD-MON-YYYY'), 28000);
INSERT INTO STAFF VALUES (seq_staff_id.NEXTVAL, 3, 'Linda Accountant', 'Finance Manager', '0813334444', TO_DATE('12-FEB-2017', 'DD-MON-YYYY'), 40000);
INSERT INTO STAFF VALUES (seq_staff_id.NEXTVAL, 4, 'Susan Organizer', 'Community Manager', '0825556666', TO_DATE('30-APR-2020', 'DD-MON-YYYY'), 35000);
```

Payment Tracking: Records show six security fee payments (some paid, one late, one pending) and three event registration payments, with amounts from R100-R200.

```
-- Insert Payments
INSERT INTO PAYMENT VALUES (seq_payment_id.NEXTVAL, 10001, 850.00, 'Security', TO_DATE('05-MAY-2023', 'DD-MON-YYYY'), 'Paid', TO_DATE('01-MAY-2023', 'DD-MON-YYYY'));
INSERT INTO PAYMENT VALUES (seq_payment_id.NEXTVAL, 10001, 1200.50, 'Maintenance', TO_DATE('05-MAY-2023', 'DD-MON-YYYY'), 'Pending', NULL);
INSERT INTO PAYMENT VALUES (seq_payment_id.NEXTVAL, 10002, 850.00, 'Security', TO_DATE('05-MAY-2023', 'DD-MON-YYYY'), 'Paid', TO_DATE('03-MAY-2023', 'DD-MON-YYYY'));
INSERT INTO PAYMENT VALUES (seq_payment_id.NEXTVAL, 10003, 850.00, 'Security', TO_DATE('05-MAY-2023', 'DD-MON-YYYY'), 'Late', NULL);
INSERT INTO PAYMENT VALUES (seq_payment_id.NEXTVAL, 10004, 850.00, 'Security', TO_DATE('05-MAY-2023', 'DD-MON-YYYY'), 'Paid', TO_DATE('04-MAY-2023', 'DD-MON-YYYY'));
INSERT INTO PAYMENT VALUES (seq_payment_id.NEXTVAL, 10005, 850.00, 'Security', TO_DATE('05-MAY-2023', 'DD-MON-YYYY'), 'Paid', TO_DATE('02-MAY-2023', 'DD-MON-YYYY'));
```

Maintenance Request: Five service requests are logged - including urgent kitchen leaks and AC repairs, routine paintwork and irrigation issues - with different completion statuses and staff assignments.


```
-- Insert Maintenance Requests
INSERT INTO MAINTENANCE_REQUEST VALUES (seq_mr_id.NEXTVAL, 10001, 'Leaking faucet in kitchen', 'Urgent', 'Completed', TO_DATE('10-APR-2023', 'DD-MON-YYYY'), NULL, NULL);
INSERT INTO MAINTENANCE_REQUEST VALUES (seq_mr_id.NEXTVAL, 10002, 'Broken gate latch', 'Emergency', 'In Progress', NULL, 5004);
INSERT INTO MAINTENANCE_REQUEST VALUES (seq_mr_id.NEXTVAL, 10003, 'Peeling paint on exterior wall', 'Routine', 'Submitted', NULL, NULL);
INSERT INTO MAINTENANCE_REQUEST VALUES (seq_mr_id.NEXTVAL, 10004, 'Air conditioning not working', 'Urgent', 'Assigned', NULL, 5004);
INSERT INTO MAINTENANCE_REQUEST VALUES (seq_mr_id.NEXTVAL, 10005, 'Garden irrigation system leak', 'Routine', 'Completed', TO_DATE('15-APR-2023', 'DD-MON-YYYY'), NULL, NULL);
```

Security Log: Three security activities are documented - routine perimeter patrols, a suspicious vehicle incident, and visitor credential verification.

```
-- Insert Security Logs
INSERT INTO SECURITY_LOG VALUES (seq_security_log_id.NEXTVAL, 5001, TO_TIMESTAMP('25-APR-2023 08:30:00', 'DD-MON-YYYY HH24:MI:SS'), 'Perimeter Patrol', 'Completed', NULL);
INSERT INTO SECURITY_LOG VALUES (seq_security_log_id.NEXTVAL, 5002, TO_TIMESTAMP('25-APR-2023 14:45:00', 'DD-MON-YYYY HH24:MI:SS'), 'Suspicious Vehicle Incident', 'In Progress', NULL);
INSERT INTO SECURITY_LOG VALUES (seq_security_log_id.NEXTVAL, 5003, TO_TIMESTAMP('26-APR-2023 10:15:00', 'DD-MON-YYYY HH24:MI:SS'), 'Visitor Credential Verification', 'Completed', NULL);
```

Events: Three events are scheduled - a Community Braai (150 capacity), Yoga Class (30 capacity), and Estate Meeting (100 capacity), with fees ranging from free to R100 per person.

```
-- Insert Events
INSERT INTO EVENT VALUES (seq_event_id.NEXTVAL, 'Community Braai', TO_DATE('20-MAY-2023', 'DD-MON-YYYY'), 'Clubhouse Lawn', 150, 'Community Services', 50.00);
INSERT INTO EVENT VALUES (seq_event_id.NEXTVAL, 'Yoga Class', TO_DATE('15-MAY-2023', 'DD-MON-YYYY'), 'Community Hall', 30, 'Fitness Instructor', 100.00);
INSERT INTO EVENT VALUES (seq_event_id.NEXTVAL, 'Estate Meeting', TO_DATE('10-MAY-2023', 'DD-MON-YYYY'), 'Conference Room', 100, 'HOA Board', 0);
```

EVENT_REGISTRATION, insert registration records into the EVENT_REGISTRATION table each representing a different event registration. They use a sequence (seq_registration_id.NEXTVAL) to generate unique registration IDs, and include details like event ID, customer ID, registration date, number of tickets, and payment status. The first two registrations are marked as "Paid", while the third is "Pending".

```
-- Insert Event Registrations
INSERT INTO EVENT_REGISTRATION VALUES (seq_registration_id.NEXTVAL, 101, 10001, TO_DATE('01-MAY-2023', 'DD-MON-YYYY'), 4, 'Paid');
INSERT INTO EVENT_REGISTRATION VALUES (seq_registration_id.NEXTVAL, 102, 10002, TO_DATE('02-MAY-2023', 'DD-MON-YYYY'), 2, 'Paid');
INSERT INTO EVENT_REGISTRATION VALUES (seq_registration_id.NEXTVAL, 103, 10003, TO_DATE('03-MAY-2023', 'DD-MON-YYYY'), 1, 'Pending');
INSERT INTO EVENT_REGISTRATION VALUES (seq_registration_id.NEXTVAL, 104, 10004, TO_DATE('04-MAY-2023', 'DD-MON-YYYY'), 3, 'Paid');
```

Event Registration Payment, insert payments records into the PAYMENT table each linked to an event registration. They use a sequence (seq_payment_id.NEXTVAL) for unique payment IDs and include details like registration ID, amount, payment type (Event_Registration), payment date, status (Paid), and processing date. All three payments are successfully processed on the same day they were made.

```
-- Insert Event Registration Payments
INSERT INTO PAYMENT VALUES (seq_payment_id.NEXTVAL, 10001, 200.00, 'Event Registration', TO_DATE('01-MAY-2023', 'DD-MON-YYYY'), 'Paid', TO_DATE('01-MAY-2023', 'DD-MON-YYYY'));
INSERT INTO PAYMENT VALUES (seq_payment_id.NEXTVAL, 10002, 100.00, 'Event Registration', TO_DATE('02-MAY-2023', 'DD-MON-YYYY'), 'Paid', TO_DATE('02-MAY-2023', 'DD-MON-YYYY'));
INSERT INTO PAYMENT VALUES (seq_payment_id.NEXTVAL, 10004, 150.00, 'Event Registration', TO_DATE('04-MAY-2023', 'DD-MON-YYYY'), 'Paid', TO_DATE('04-MAY-2023', 'DD-MON-YYYY'));
```

Facility, insert facilities records into the FACILITY table representing a different facility in a community or recreational space. They use a sequence (seq_facility_id.NEXTVAL) to generate unique facility IDs and include details like the facility name, type, usage fee (where 0 means free), and maintenance schedule. The facilities include a free-swimming pool with weekly cleaning, a paid tennis court with monthly inspections, a clubhouse with a higher fee and bi-weekly maintenance, and a free gym with daily cleaning.

```
-- Insert Facilities
INSERT INTO FACILITY VALUES (seq_facility_id.NEXTVAL, 'Swimming Pool', 'Recreational', 0, 'Weekly cleaning on Mondays');
INSERT INTO FACILITY VALUES (seq_facility_id.NEXTVAL, 'Tennis Court', 'Sports', 50, 'Monthly inspection');
INSERT INTO FACILITY VALUES (seq_facility_id.NEXTVAL, 'Clubhouse', 'Community', 200, 'Bi-weekly maintenance');
INSERT INTO FACILITY VALUES (seq_facility_id.NEXTVAL, 'Gym', 'Recreational', 0, 'Daily cleaning');
```

Bookings, insert booking records into the BOOKING table, each with a unique booking ID generated by the sequence seq_booking_id.NEXTVAL. Each booking includes details such as the facility ID, customer ID, booking date, time slot (Morning/Afternoon/Evening), and status (Completed/Confirmed). The bookings show different facilities being reserved by various customers across multiple dates, with some sessions already completed and others confirmed for future use.

```
-- Insert Bookings
INSERT INTO BOOKING VALUES (seq_booking_id.NEXTVAL, 1001, 10001, TO_DATE('01-MAY-2023', 'DD-MON-YYYY'), 'Morning', 'Completed');
INSERT INTO BOOKING VALUES (seq_booking_id.NEXTVAL, 1002, 10002, TO_DATE('03-MAY-2023', 'DD-MON-YYYY'), 'Afternoon', 'Confirmed');
INSERT INTO BOOKING VALUES (seq_booking_id.NEXTVAL, 1003, 10003, TO_DATE('05-MAY-2023', 'DD-MON-YYYY'), 'Evening', 'Confirmed');
INSERT INTO BOOKING VALUES (seq_booking_id.NEXTVAL, 1001, 10004, TO_DATE('02-MAY-2023', 'DD-MON-YYYY'), 'Morning', 'Completed');
```

Complaint, insert records into the COMPLAINT table, each representing a different complaint from residents. They use a sequence (seq_complaint_id.NEXTVAL) to generate unique complaint IDs and include details such as the resident ID, complaint category, description, status (Open/In Progress/Resolved), and submission date. The complaints range from a security issue (rude behavior) to maintenance delays and cleanliness concerns about the clubhouse, showing different stages of resolution.


```
-- Insert Complaints
INSERT INTO COMPLAINT VALUES (seq_complaint_id.NEXTVAL, 200001, 1, 'Security guard was rude during night patrol', 'Open', TO_DATE('24-APR-2023', 'DD-MON-YYYY'));
INSERT INTO COMPLAINT VALUES (seq_complaint_id.NEXTVAL, 200003, 2, 'Maintenance request from last week still not addressed', 'In Progress', TO_DATE('26-APR-2023', 'DD-MON-YYYY'));
INSERT INTO COMPLAINT VALUES (seq_complaint_id.NEXTVAL, 200006, 4, 'Clubhouse was left dirty after event', 'Resolved', TO_DATE('20-APR-2023', 'DD-MON-YYYY'));
```

Budget, insert records into the BUDGET table, each representing an annual budget allocation for different departments or categories. They use a sequence (seq_budget_id.NEXTVAL) to generate unique budget IDs and include details like department ID, fiscal year (2023-24), allocated amount, and approval status (Approved/Pending). The budgets show varying allocations (ranging from 300,000 to 500,000), with most approved except for department 3's which remains pending.

```
-- Insert Budgets
INSERT INTO BUDGET VALUES (seq_budget_id.NEXTVAL, 1, '2023-24', 500000, 'Approved');
INSERT INTO BUDGET VALUES (seq_budget_id.NEXTVAL, 2, '2023-24', 450000, 'Approved');
INSERT INTO BUDGET VALUES (seq_budget_id.NEXTVAL, 3, '2023-24', 300000, 'Pending');
INSERT INTO BUDGET VALUES (seq_budget_id.NEXTVAL, 4, '2023-24', 350000, 'Approved');
```

RECYCLING_LOG, insert records into the RECYCLING_LOG table, tracking recycling activities by different residents. Each entry uses a sequence (seq_recycling_log_id.NEXTVAL) to generate a unique log ID and includes resident ID, recycling date (all on 24-APR-2023), weight of recycled materials (ranging from 5.5 to 15.0 units), and material type (Paper, Plastic, Glass, or Metal). The logs show residents recycling different materials in varying quantities on the same day.

```
-- Insert Recycling Logs
INSERT INTO RECYCLING_LOG VALUES (seq_recycling_log_id.NEXTVAL, 10001, TO_DATE('24-APR-2023', 'DD-MON-YYYY'), 12.5, 'Paper');
INSERT INTO RECYCLING_LOG VALUES (seq_recycling_log_id.NEXTVAL, 10002, TO_DATE('24-APR-2023', 'DD-MON-YYYY'), 8.75, 'Plastic');
INSERT INTO RECYCLING_LOG VALUES (seq_recycling_log_id.NEXTVAL, 10003, TO_DATE('24-APR-2023', 'DD-MON-YYYY'), 15.0, 'Glass');
INSERT INTO RECYCLING_LOG VALUES (seq_recycling_log_id.NEXTVAL, 10004, TO_DATE('24-APR-2023', 'DD-MON-YYYY'), 5.5, 'Metal');
```

SAFETY_INSPECTION, insert records into the SAFETY_INSPECTION table, documenting safety inspections conducted at facility ID 5001. Each record uses a sequence (seq_inspection_id.NEXTVAL) to generate a unique inspection ID and includes:

1. The facility ID (5001 for both), inspection date (15-APR-2023 and 20-APR-2023), identified issues (e.g., "Fire extinguishers need servicing"), actions taken (e.g., "Contacted service provider"), and a follow-up date (15-JUL-2023 and 20-JUL-2023).
2. The first inspection flagged fire safety concerns, while the second noted a tripping hazard near the pool, both with planned resolutions.
3. The follow-up dates suggest a 3-month review period for each issue.

```
-- Insert Safety Inspections
INSERT INTO SAFETY_INSPECTION VALUES (seq_inspection_id.NEXTVAL, 5001, TO_DATE('15-APR-2023', 'DD-MON-YYYY'), 'Fire extinguishers need servicing', 'Contacted service provider');
INSERT INTO SAFETY_INSPECTION VALUES (seq_inspection_id.NEXTVAL, 5001, TO_DATE('20-APR-2023', 'DD-MON-YYYY'), 'Uneven pavement near pool area', 'Scheduled repair for next week');
```

UTILITY, insert utility billing records into the **UTILITY** table, tracking consumption for different residents. Each entry uses a sequence (seq_utility_id.NEXTVAL) to generate a unique ID and includes:

1. **Resident ID** (10001, 10002, 10003), **utility type** (Water/Electricity), **usage/cost** (e.g., 12.5 units for water, \$325.75 for electricity), and **billing period** (all for April 2023).
2. The data shows varying consumption patterns—for example, resident 10003 has higher electricity usage (\$410) compared to others, while water usage ranges from 9.8 to 15.2 units.
3. All entries are for the same billing cycle (2023-04), suggesting a monthly utility tracking system for water and electricity.

```
-- Insert Utilities
INSERT INTO UTILITY VALUES (seq_utility_id.NEXTVAL, 10001, 'Water', 12.5, '2023-04');
INSERT INTO UTILITY VALUES (seq_utility_id.NEXTVAL, 10001, 'Electricity', 325.75, '2023-04');
INSERT INTO UTILITY VALUES (seq_utility_id.NEXTVAL, 10002, 'Water', 9.8, '2023-04');
INSERT INTO UTILITY VALUES (seq_utility_id.NEXTVAL, 10002, 'Electricity', 280.5, '2023-04');
INSERT INTO UTILITY VALUES (seq_utility_id.NEXTVAL, 10003, 'Water', 15.2, '2023-04');
INSERT INTO UTILITY VALUES (seq_utility_id.NEXTVAL, 10003, 'Electricity', 410.0, '2023-04');
```

3.5 Sequences

These commands create unique numbering sequences for database tables, each starting at a specific value and incrementing by 1. They ensure automatic ID generation for households, residents, staff, payments, maintenance requests, security logs, events, facilities, bookings, complaints, budgets, recycling logs, inspections, utilities, and event registrations.

```

=====
-- SEQUENCES FOR AUTO-INCREMENTING IDs
=====

CREATE SEQUENCE seq_household_id START WITH 10001 INCREMENT BY 1;
CREATE SEQUENCE seq_resident_id START WITH 200001 INCREMENT BY 1;
CREATE SEQUENCE seq_staff_id START WITH 5001 INCREMENT BY 1;
CREATE SEQUENCE seq_payment_id START WITH 1000001 INCREMENT BY 1;
CREATE SEQUENCE seq_mr_id START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_security_log_id START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_event_id START WITH 101 INCREMENT BY 1;
CREATE SEQUENCE seq_facility_id START WITH 1001 INCREMENT BY 1;
CREATE SEQUENCE seq_booking_id START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_complaint_id START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_budget_id START WITH 1001 INCREMENT BY 1;
CREATE SEQUENCE seq_recycling_log_id START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_inspection_id START WITH 1001 INCREMENT BY 1;
CREATE SEQUENCE seq_utility_id START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_registration_id START WITH 1 INCREMENT BY 1;

```

4. Queries

4.1 Limitation of rows and columns

```

-- 1. Query with LIMITATION OF ROWS AND COLUMNS
-- Show top 3 households with highest utility consumption
SELECT * FROM (
    SELECT h.Household_ID, h.Primary_Contact, u.Type, u.Usage_Value
    FROM HOUSEHOLD h
    JOIN UTILITY u ON h.Household_ID = u.Household_ID
    WHERE u.Month = '2023-04'
    ORDER BY u.Usage_Value DESC
) WHERE ROWNUM <= 3;

```

4.2 Sorting

```

-- 2. Query with SORTING
-- List all events sorted by date and then by name
SELECT Event_ID, Name, Event_Date, Location, Capacity
FROM EVENT
ORDER BY Event_Date, Name;

```

4.3 LIKE, AND and OR

```

-- 3. Query with LIKE, AND, OR
-- Find maintenance requests containing 'leak' or 'broken' that are urgent or emergency
SELECT mr.Request_ID, h.Primary_Contact, mr.Description, mr.Priority, mr.Status
FROM MAINTENANCE_REQUEST mr
JOIN HOUSEHOLD h ON mr.Household_ID = h.Household_ID
WHERE (mr.Description LIKE '%leak%' OR mr.Description LIKE '%broken%')
AND (mr.Priority = 'Urgent' OR mr.Priority = 'Emergency');

```


4.4 Variables and character functions

```
-- 4. Query with VARIABLES and CHARACTER FUNCTIONS
-- Generate resident directory with formatted names and contact info
SELECT
    INITCAP(r.Name) AS Resident_Name,
    h.Primary_Contact AS Household_Contact,
    '(' || SUBSTR(r.Contact_Number, 1, 3) || ')' || ' ' ||
    SUBSTR(r.Contact_Number, 4, 3) || '-' ||
    SUBSTR(r.Contact_Number, 7) AS Formatted_Phone,
    LOWER(r.Email) AS Email
FROM RESIDENT r
JOIN HOUSEHOLD h ON r.Household_ID = h.Household_ID;
```

4.5 Round or trunc

```
-- 5. Query with ROUND or TRUNC
-- Calculate average recycling per household with rounding
SELECT
    h.Household_ID,
    h.Primary_Contact,
    ROUND(AVG(rl.Weight), 2) AS Avg_Recycling_Weight,
    COUNT(rl.Log_ID) AS Collections
FROM HOUSEHOLD h
JOIN RECYCLING_LOG rl ON h.Household_ID = rl.Household_ID
GROUP BY h.Household_ID, h.Primary_Contact;
```

4.6 Date functions

```
-- 6. Query with DATE FUNCTIONS
-- Find payments due in the next 7 days
SELECT
    Payment_ID,
    Household_ID,
    Amount,
    Due_Date,
    ROUND(Due_Date - SYSDATE) AS Days_Until_Due
FROM PAYMENT
WHERE Status = 'Pending'
AND Due_Date BETWEEN SYSDATE AND SYSDATE + 7;
```

4.7 Aggregate functions

```
-- 7. Query with AGGREGATE FUNCTIONS
-- Calculate department budget utilization
SELECT
    d.Name AS Department,
    COUNT(s.Staff_ID) AS Staff_Count,
    SUM(s.Salary) AS Total_Salary,
    b.Total_Amount AS Budget_Allocation,
    ROUND(SUM(s.Salary)/b.Total_Amount*100, 2) AS Salary_Percentage
FROM DEPARTMENT d
JOIN STAFF s ON d.Department_ID = s.Department_ID
JOIN BUDGET b ON d.Department_ID = b.Department_ID
GROUP BY d.Name, b.Total_Amount;
```

4.8 Group by and having

```
-- 8. Query with GROUP BY and HAVING
-- Find households with more than 1 resident
SELECT
    h.Household_ID,
    h.Primary_Contact,
    COUNT(r.Resident_ID) AS Resident_Count
FROM HOUSEHOLD h
JOIN RESIDENT r ON h.Household_ID = r.Household_ID
GROUP BY h.Household_ID, h.Primary_Contact
HAVING COUNT(r.Resident_ID) > 1;
```

4.9 Joins

```
-- 9. Query with JOINS
-- Show all maintenance requests with assigned staff and household info
SELECT
    mr.Request_ID,
    h.Primary_Contact,
    mr.Description,
    mr.Priority,
    mr.Status,
    s.Name AS Assigned_Staff,
    d.Name AS Department
FROM MAINTENANCE_REQUEST mr
JOIN HOUSEHOLD h ON mr.Household_ID = h.Household_ID
LEFT JOIN STAFF s ON mr.Assigned_Staff_ID = s.Staff_ID
LEFT JOIN DEPARTMENT d ON s.Department_ID = d.Department_ID;
```

4.10 Sub-queries

```
-- 10. Query with SUB-QUERIES
-- Find households that haven't submitted any maintenance requests
SELECT
    h.Household_ID,
    h.Primary_Contact,
    h.Property_Size
FROM HOUSEHOLD h
WHERE h.Household_ID NOT IN (
    SELECT DISTINCT Household_ID
    FROM MAINTENANCE_REQUEST
);
```

NEW QUERY: List all events with available spots

```
-- 11. NEW QUERY: List all events with available spots
SELECT e.Event_ID, e.Name, e.Event_Date, e.Location,
       e.Capacity - NVL(SUM(er.Number_Of_Attendees), 0) AS Available_Spots
FROM EVENT e
LEFT JOIN EVENT_REGISTRATION er ON e.Event_ID = er.Event_ID
GROUP BY e.Event_ID, e.Name, e.Event_Date, e.Location, e.Capacity
HAVING e.Capacity - NVL(SUM(er.Number_Of_Attendees), 0) > 0;
```

NEW QUERY: Show total event revenue by department

```
-- 12. NEW QUERY: Show total event revenue by department
SELECT
    d.Name AS Department,
    SUM(e.Cost_Per_Person * er.Number_Of_Attendees) AS Total_Revenue,
    COUNT(DISTINCT e.Event_ID) AS Number_Of_Events
FROM EVENT e
JOIN EVENT_REGISTRATION er ON e.Event_ID = er.Event_ID
JOIN DEPARTMENT d ON d.Name = e.Organizer
WHERE er.Payment_Status = 'Paid'
GROUP BY d.Name;
```